The Impossible Archive Engine

Implementation-Ready Algorithm Plan

Algorithmic Implementation Architect

May 3, 2026

Version: 1.0
Status: Final architecture package
Purpose: Design a simulated historiographic engine with hidden truth, biased evidence, unreliable interpretation, progressive discovery, and mystery preservation.

Contents

---

1. Executive Summary

The Impossible Archive Engine is not a fantasy lore generator. It is a simulated historiographic system.

The engine generates a fictional civilization that never existed, maintains a private causal history, and produces public-facing archival fragments that are incomplete, biased, contradictory, damaged, forged, mistranslated, or ritually non-factual. It then simulates interpreters who build changing theories from the visible archive.

The central architectural rule is:

> Generate and preserve a layered hidden truth model first, then generate public evidence as historically situated projections, distortions, silences, rituals, and misreadings of that model.

The stronger final target is not merely:

hidden truth graph -> artifact generator -> interpreter engine

It is:

layered truth model
-> hidden causal world
-> culturally specific evidence practices
-> artifact voice compiler
-> contradiction and anachronism validator
-> mystery preservation engine
-> epistemic interpreter simulation
-> access-filtered historiography

The result should feel less like a fantasy encyclopedia and more like a damaged archive whose truth is mediated by ritual, fraud, politics, language drift, decay, scholarship, and loss.

---

2. Clarified Problem Statement

2.1 Goal

Build a system called **The Impossible Archive Engine** that can:

1. Generate a coherent hidden history for an imaginary civilization across centuries.
2. Generate discovered public artifacts from that history.
3. Ensure each artifact reflects its creator's era, knowledge, bias, motive, language, and material conditions.
4. Simulate historians, archaeologists, religious authorities, propagandists, curators, philologists, and conspiracy theorists interpreting the archive.
5. Maintain consistency in the hidden truth model while allowing public evidence to contradict itself for explicit historical reasons.
6. Support progressive discovery without rewriting hidden truth.
7. Preserve mysteries where appropriate instead of over-explaining every unknown.
8. Enforce access control between public evidence, scholarly inference, curator knowledge, canonical truth, and debug traces.

2.2 Non-Goals

The system should not be:

- a random name generator,
- a generic fantasy timeline generator,
- a simple knowledge graph visualizer,
- a perfect canonical encyclopedia,
- or a system where every contradiction is treated as a bug.

Public contradictions are allowed, but accidental contradictions are not.

2.3 Core Design Constraint

Every public-facing fragment must be traceable to at least one of the following:

- hidden canonical truth,
- partial knowledge,
- bias,
- damage,
- mistranslation,
- copying error,
- calendar conversion error,
- propaganda,
- censorship,
- forgery,
- ritual convention,
- mythic compression,
- meaningful silence,
- genuine uncertainty,
- protected mystery,
- or explicit generation bug.

---

3. Explicit Inputs and Outputs

3.1 Initial World Generation Inputs

Input	Description
seed	Deterministic seed for replayable generation.
civilization_scope	Time span, geographic scale, population scale, cultural complexity.
historical_density	Approximate number of major events per century.
artifact_density	Expected number of discovered artifacts per era, site, faction, or theme.
strangeness_profile	Desired novelty, surrealism, taboo structure, and ecological distinctiveness.
truth_policy	Rules for canonical truth, unresolved truth, mythic truth, and access.
mystery_policy	Which topics may be fully solved, partially solved, or permanently ambiguous.
language_depth	Lightweight, medium, or deep simulation of language drift.

Input	Description
interpreter_profiles	Types of interpreters to simulate.
real_world_similarity_limits	Thresholds against direct copies of familiar civilizations.
runtime_mode	In-memory prototype, file-backed engine, or database-backed engine.

3.2 Runtime Query Inputs

A user may request:

Request Type	Example
Artifact	Generate a newly discovered inscription.
Exhibit	Create a museum exhibit for the Reservoir Gate ruins.
Translation	Show a disputed translation of a ritual song.
Timeline	Show the public timeline of the Salt-Moon Schism.
Interpretation	What would a scholar in year 810 believe?
Debate	Generate a debate between two historians.
Reconstruction	Reconstruct a ruined city from available evidence.
Hidden truth	What really happened?
Audit	Which artifacts are forged?
Contradiction review	What contradictions remain unresolved?

Each query should include or derive:

user_access_level
current_archive_date
requested_output_format
topic_or_entity_scope
visibility_scope

3.3 Outputs

Public Outputs

- artifact descriptions,

- museum labels,
- public timelines,
- disputed translations,
- debates between interpreters,
- maps with uncertain legends,
- scholar theories,
- archaeological reports,
- reconstruction summaries,
- unresolved mystery summaries.

Restricted Outputs

- true event graph,
- actual artifact provenance,
- known forgeries,
- hidden causal links,
- true contradiction causes,
- generation traces,
- validation logs,
- unresolved generation bugs.

Internal Outputs

- validation reports,
- contradiction ledger,
- anachronism reports,
- access-filter traces,
- originality scores,
- deterministic replay logs,
- theory revision history.

---

4. Core Architecture

4.1 System Components

Component	Responsibility
Hidden Truth Graph	Maintains canonical events, entities, causal links, and layered truth.
Public Archive Graph	Stores artifacts, claims, citations, contradictions, theories, and discoveries.
Artifact Simulator	Produces public fragments from hidden truth using bias, motive, damage, language, and genre.

Component	Responsibility
Artifact Voice Compiler	Renders artifact text in historically specific forms and registers.
Contradiction Manager	Detects contradictions and assigns causes.
Anachronism Detector	Flags references unavailable at a claimed date.
Language Drift Engine	Evolves names, titles, scripts, terms, dialects, and translation traps.
Mystery Preservation Engine	Prevents over-resolution of protected mysteries.
Originality Pressure Engine	Detects familiar structures and forces civilization-specific transformations.
Interpreter Engine	Simulates epistemic styles and evolving theories.
Discovery Engine	Introduces new artifacts and updates public interpretations.
Access Control Layer	Filters truth, evidence, and audit data based on permissions.

4.2 Main Data Flow

configuration + seed
-> generate geography and ecological pressures
-> generate hidden entities and events
-> validate causal truth graph
-> generate artifact origins and possible evidence traces
-> discover artifacts over time
-> extract public claims
-> detect anachronisms and contradictions
-> assign contradiction causes
-> update interpreters and theories
-> answer user queries through access filters

4.3 Key Invariants

Invariant	Requirement
Hidden causality	Causes must precede effects.
Entity existence	Events and artifacts cannot reference nonexistent entities unless mediated by myth, error, fraud, or ritual.
Public contradiction	Allowed only with explicit cause.
Artifact provenance	Every artifact must have required metadata.
Theory citation	Every interpretation must cite visible

Invariant	Requirement
	artifacts.
Hidden truth isolation	Public reinterpretation must never mutate canonical history.
Deterministic replay	Same seed and config reproduce the same world state.
Access safety	Public users cannot receive hidden truth unless supported by public evidence.

5. Hidden-State Model

The hidden state is a private causal knowledge graph. It should not be stored as prose. Prose is a rendered view over structured state.

5.1 Core Entity Types

Civilization
Region
Settlement
Site
Dynasty
Faction
Religion
Sect
Person
Office
Language
Dialect
Script
Technology
TradeGood
Route
Event
Disaster
Migration
War
Law
MythSource
EcologicalChange
CulturalPractice
Taboo

ArtifactOrigin
Mystery

5.2 Hidden Entity Schema

Entity:
- id
- type
- canonical_name
- alternate_names
- existence_interval
- location_interval
- created_by_event_id
- ended_by_event_id
- properties
- access_tags

5.3 Hidden Event Schema

Event:
- event_id
- event_type
- start_date
- end_date
- location_ids
- participants
- causes
- effects
- created_entities
- modified_entities
- destroyed_entities
- related_events
- canonical_description
- certainty_level
- truth_layer
- access_tags

5.4 Hidden-State Invariants

Invariant	Rule
Cause order	cause.end_date <= event.start_date
Lifespan validity	Participants must exist during the event unless the event is ritual, mythic, forged, or symbolic.
Technology validity	Required technologies must exist and be reachable.

Invariant	Rule
Location validity	Locations must exist and be accessible.
Causal closure	Major state changes must have causes or be marked as unknowable/lost.
No accidental contradiction	Canonical truth cannot contradict itself accidentally.
Versioning	Hidden truth changes require explicit migration records.

6. Layered Truth and Mystery Model

The engine should not treat truth as a single clean layer. A perfect hidden answer for every question can make the archive feel less mysterious.

6.1 Truth Layers

Layer	Meaning	Example
canonical_truth	What actually happened in the hidden history.	The drought was caused by reservoir mismanagement.
engine_known_truth	What the engine knows internally and may expose to privileged access.	The engine knows who forged a decree.
protected_mystery	A deliberately unresolved area.	No final answer exists for what the Ash Oracle originally meant.
mythic_truth	Not factual, but culturally operative.	The city believes it was founded by a dead river.
ritual_truth	Valid because performed, not because factual.	A dead king signs decrees through mortuary office.
public_inference	What scholars currently infer from evidence.	The exile migration likely occurred around year 610.
forged_truth	A false claim designed to become socially real.	A forged genealogy legitimizes a dynasty.
unknowable_loss	Evidence is destroyed beyond recovery.	The original name of the first capital is unrecoverable.
generation_bug	Accidental contradiction requiring repair.	A title appears too early without explanation.

6.2 Epistemic Truth Schema

EpistemicTruth:
  truth_id

subject
truth_layer
certainty_level
knowability_status
hidden_resolution
public_resolution
protected_from_resolution
allowed_interpretive_range
evidence_policy
access_tags

6.3 Mystery Schema

Mystery:
mystery_id
title
subject_entities
origin_event_ids
mystery_type
protected_question
allowed_answers
forbidden_answers
current_public_theories
clue_artifact_ids
misleading_artifact_ids
reveal_policy
preservation_policy
entropy_budget
access_tags

6.4 Mystery Types

Type	Description
identity_mystery	Who was this author, ruler, saint, founder, or traitor?
event_mystery	What happened during a disputed event?
location_mystery	Where was a city, tomb, shrine, route, or battlefield?
semantic_mystery	What did a word, symbol, title, or ritual phrase mean?
chronology_mystery	When did something happen?
authenticity_mystery	Is an artifact genuine, copied, forged, interpolated, or misdated?
ritual_paradox	Contradiction preserved because it has

Type	Description
	sacred or legal force.
lost_cause	Causal gap that cannot be reconstructed.
archive_silence	Meaningful absence caused by censorship, taboo, decay, or non-recording.

6.5 Reveal Policy

```
RevealPolicy:
  reveal_mode:
    - fully_resolvable
    - partially_resolvable
    - never_fully_resolvable
    - access_locked
    - contradictory_by_design
    - resolved_only_in_mythic_terms
  max_confidence_public_theory
  max_confidence_scholar_theory
  allowed_new_clues
  forbidden_clue_types
  can_hidden_truth_answer_directly
```

6.6 Mystery Preservation Rule

If a new artifact would over-resolve a protected mystery, the system must either:

1. cap confidence,
2. introduce interpretive ambiguity,
3. reveal that the artifact answers a different question,
4. mark the artifact as suspicious,
5. or classify the mystery as intentionally resolved only for privileged access.

7. Public Archive Model

The public archive is a graph of discovered evidence and interpretations. It is not the truth.

7.1 Public Archive Graph Contents

Artifact nodes
Claim nodes
Interpretation nodes
Theory nodes
Contradiction nodes
Discovery nodes

Translation nodes
Mystery nodes
Citation edges
Support edges
Dispute edges
Forgery suspicion edges
Access-filtered projections

7.2 Public Claim Schema

Claim:
- claim_id
- source_artifact_id
- claim_type
- subject
- predicate
- object
- literal_content
- intended_function
- factuality_status
- ritual_validity
- legal_validity
- symbolic_mapping
- confidence
- access_tags

7.3 Claim Types

Claim Type	Description
factual_claim	Intended as a factual statement.
legal_fiction	Literally false but institutionally valid.
ritual_claim	Meaningful because performed.
mythic_compression	Many people or events collapsed into one story.
taboo_substitution	Forbidden name or event replaced with permitted symbol.
propaganda_claim	Distortion motivated by power.
mourning_formula	Conventional grief language mistaken as fact.
anti_record	Record designed to erase or misdirect.
negative_evidence	Meaningful absence, omission, silence, or erasure.
translation_guess	Scholar reconstruction from uncertain language.

7.4 Public Archive Invariants

Invariant	Requirement
Source requirement	Every claim must cite an artifact.
Contradiction requirement	Every contradiction must have an assigned cause or be marked as generation bug.
Interpretation requirement	Every theory must cite visible evidence.
Provenance requirement	Every artifact must have required metadata.
Access requirement	Public archive views must be filtered by user access.
Canon isolation	Public theories must not rewrite hidden truth.

8. Artifact Data Model

Artifacts are public evidence objects generated from hidden truth, distortion, material conditions, voice rules, and discovery context.

8.1 Artifact Schema

Artifact:
- artifact_id
- artifact_type
- title
- creator_id
- attributed_creator_id
- true_creation_date
- claimed_creation_date
- discovery_date
- location_created
- location_found
- original_language_id
- dialect_id
- script_id
- material
- preservation_quality
- damage_profile
- transmission_history
- creator_knowledge_scope
- creator_bias_profile
- creator_motive
- intended_audience
- public_text

literal_translation
scholarly_translation
claims
hidden_truth_links
distortion_profile
reliability_score
contradiction_links
mystery_links
access_tags
generation_trace

8.2 Required Artifact Metadata

Field	Required
Artifact ID	Yes
Creator or attributed creator	Yes
True creation date	Yes
Claimed creation date	Yes
Discovery date	Yes
Location found	Yes
Language or dialect	Yes
Preservation quality	Yes
Bias profile	Yes
Reliability score	Yes
Relation to hidden truth	Yes
Contradiction links	Yes
Generation trace	Yes for debug/curator modes

8.3 Artifact Type Profiles

Artifact Type	Likely Distortions
Inscription	Formulaic language, erosion, elite propaganda, missing names.
Damaged manuscript	Lacunae, copy errors, interpolations, harmonization.
Oral history	Mythic compression, heroic bias, temporal collapse.
Forged decree	Anachronistic title, false seal, political motive.
Misdated ruin	Stratigraphy error, reused stones, intrusive burial.
Ritual song	Symbolic language, repetition, archaic terms,

Artifact Type	Likely Distortions
	theological layering.
Trade ledger	Numeric reliability, narrow scope, place-name variants.
Burial object	Iconographic ambiguity, elite bias, ritual inversion.
Scholar commentary	Citations, disciplinary assumptions, inherited errors.
Map with errors	Obsolete borders, copied coastlines, deliberate omissions.

8.4 Reliability Score

Reliability should be computed from components rather than treated as a vague number.

```
reliability_score = weighted_average(
  provenance_confidence,
  preservation_integrity,
  temporal_proximity,
  creator_access_to_events,
  bias_penalty,
  forgery_penalty,
  translation_confidence,
  external_corroboration,
  contradiction_penalty
)
```

---

9. Artifact Voice Engine

The artifact generator must not merely produce metadata-correct summaries. It must produce artifact prose that feels historically specific.

9.1 Voice Profile Schema

```
ArtifactVoiceProfile:
  artifact_type
  era
  region
  language_snapshot
  social_register
  institutional_context
  taboo_constraints
```

formulae
allowed_metaphors
forbidden_terms
scribal_habits
abbreviation_rules
damage_rules
translation_style
cataloging_style

9.2 Voice Dimensions

Dimension	Examples
social_register	royal, priestly, mercantile, funerary, domestic, military, legal
textual_form	decree, lament, ledger, curse, oath, itinerary, inventory, marginal note
rhetorical_mode	repetitive, elliptical, accusatory, euphemistic, ecstatic, bureaucratic
damage_mode	lacunae, broken line, missing determinative, overwritten name, water loss
translation_apparatus	brackets, uncertain readings, alternate glosses, notes
taboo_handling	name omission, symbol substitution, reverse writing, deliberate silence
scribal_behavior	abbreviation, correction, gloss, copying error, harmonization
catalog_bias	colonial, nationalist, sectarian, antiquarian, museum-neutral

9.3 Artifact Voice Pipeline

```
function render_artifact_voice(artifact, hidden_truth, public_archive):
  voice_profile = select_voice_profile(
    artifact.type,
    artifact.true_creation_date,
    artifact.creator,
    artifact.language_snapshot,
    artifact.institutional_context
  )

  semantic_payload = select_claims_and_omissions(artifact)

  culturally_filtered_payload = apply_taboo_and_formula_rules(
    semantic_payload,
```

```
    voice_profile
)

register_text = render_in_register(
    culturally_filtered_payload,
    voice_profile
)

damaged_text = apply_preservation_damage(
    register_text,
    artifact.damage_profile
)

translated_text = apply_translation_layer(
    damaged_text,
    artifact.translation_context,
    voice_profile
)

catalog_text = apply_modern_catalog_bias(
    translated_text,
    artifact.discovery_context
)

return catalog_text
```

9.4 Artifact Quality Criterion

A generated artifact is too generic if it could be moved to a different civilization without changing its language, institutional assumptions, taboos, or material context.

---

10. Interpreter and Scholar Model

Interpreters are not neutral summarizers. They are agents with methods, priors, incentives, access limits, and epistemic styles.

10.1 Interpreter Types

Historian
Archaeologist
PoliticalPropagandist
ReligiousAuthority
ConspiracyTheorist

MuseumCurator
Philologist
ColonialAdministrator
LocalOralHistorian
ForgerySpecialist
RuinSurveyor
TradeHistorian

10.2 Interpreter Schema

Interpreter:
- interpreter_id
- name
- type
- active_date_range
- institutional_affiliation
- access_level
- known_artifact_ids
- methodological_profile
- epistemic_style
- ideological_biases
- taboo_constraints
- language_competence
- trust_model
- theory_preferences
- reputation
- risk_tolerance
- susceptibility_to_forgery

10.3 Epistemic Styles

Style	Behavior
stratigraphic_literalist	Trusts physical layers over texts.
heresy_harmonizer	Reconciles contradictions into theology.
anti_dynastic_revisionist	Treats royal inscriptions as hostile evidence.
philological_maximalist	Reconstructs history from sound changes and scribal forms.
ritual_formalist	Treats factual contradiction as irrelevant if ritually valid.
archive_conspiracist	Treats missing evidence as deliberate proof.
bureaucratic_minimalist	Trusts ledgers, seals, and mundane records over myths.
catastrophist	Interprets cultural shifts as disaster aftermaths.

Style	Behavior
continuity_defender	Explains disruption as surface variation over stable tradition.
rupture_theorist	Sees sharp breaks, invasions, revolutions, and replacements.

10.4 Theory Schema

Theory:
  theory_id
  interpreter_id
  created_date
  last_updated_date
  theory_claims
  cited_artifact_ids
  supporting_claim_ids
  contradicting_claim_ids
  ignored_claim_ids
  confidence
  public_status
  revision_history

10.5 Theory Scoring

theory_score =
  evidence_support_score
  - contradiction_penalty
  + prior_alignment_bonus
  + institutional_incentive_bonus
  - complexity_penalty
  - ignored_evidence_penalty

The weightings depend on the interpreter. A propagandist may value political utility; a philologist may value script and sound change; a ritual formalist may treat factual contradictions as secondary.

---

11. Timeline and Causality Model

11.1 Timeline Data

Timeline:
  calendar_systems
  eras
  events

entity_lifespans
language_snapshots
border_snapshots
technology_availability
religious_term_availability
place_name_history

11.2 Event DAG

Events form a directed causal graph:

causes -> event -> effects

Example:

Drought
-> canal tax reform
-> temple revolt
-> religious schism
-> exile migration
-> dialect split
-> later mistranslation of exile songs

11.3 Causality Edge Types

Edge Type	Meaning
causes	Direct causal driver.
enables	Makes later event possible.
prevents	Blocks or suppresses another event.
reacts_to	Response to an earlier event.
mythologizes	Later cultural reinterpretation.
censors	Deliberate erasure or modification.
copies	Textual or artifact transmission.
misdates	Public date differs from true date.
forges	False artifact claims origin it lacks.
translates_as	Linguistic transformation.

11.4 Calendar System Schema

CalendarSystem:
calendar_id
name
start_epoch
month_structure

leap_rules
religious_adjustments
political_reforms
known_conversion_errors

Calendar conversion errors are valid causes of public contradictions.

---

12. Contradiction Taxonomy

12.1 Contradiction Schema

Contradiction:
contradiction_id
involved_claim_ids
involved_artifact_ids
contradiction_type
assigned_cause
hidden_truth_resolution
public_resolution_status
detected_date
access_tags

12.2 Contradiction Types

Type	Example
Date contradiction	Two artifacts assign different dates to the same war.
Person contradiction	A person appears in an event after death.
Place contradiction	A city is named before its founding.
Title contradiction	A title appears before the office exists.
Language contradiction	A word appears before that dialect developed it.
Technology contradiction	Material or tool appears before the technology exists.
Religious contradiction	Sectarian doctrine appears before the schism.
Genealogy contradiction	Two incompatible parentage claims.
Route contradiction	Trade route crosses an impassable region.
Map contradiction	Borders reflect a later political era.
Script contradiction	Artifact uses a later script form.
Ritual contradiction	Factually impossible but ritually valid.

12.3 Allowed Contradiction Causes

Forgery
Mistranslation
Damage
CalendarConversionError
Propaganda
MythologizedMemory
DeliberateCensorship
GenuineUncertainty
LaterCopy
InterpolatedText
MisdatedStratum
RitualAnachronism
PropheticConvention
CopyistError
CompositeArtifact
ProtectedMystery
UnknowableLoss
UnresolvedGenerationBug

12.4 Contradiction Rule

```
if contradiction.assigned_cause is null:
    reject_or_mark_generation_bug(contradiction)
```

---

13. Anachronism Detection Strategy

13.1 Definition

An artifact has an anachronism if its claimed creation date includes a reference unavailable at that date.

This includes:

events
people
places
titles
laws
technologies
trade goods
religious concepts
script forms

language forms
place names
political borders
calendar systems
mythic conventions

13.2 Detection Inputs

artifact.claimed_creation_date
artifact.true_creation_date
artifact.text_claims
artifact.language_id
artifact.dialect_id
artifact.script_id
artifact.material
artifact.referenced_entities
artifact.transmission_history
artifact.distortion_profile

13.3 Detection Algorithm

1. Extract referenced entities, terms, dates, titles, technologies, materials, places, and scripts.
2. Resolve each reference to hidden truth graph IDs where possible.
3. Check each reference against the claimed creation date.
4. If unavailable, check whether a valid mediating explanation exists:
   - later copy,
   - forgery,
   - misdating,
   - interpolation,
   - ritual convention,
   - prophetic convention,
   - composite artifact,
   - calendar conversion error.
5. If no explanation exists, flag accidental anachronism.
6. Reject, repair, or mark as generation bug depending on mode.

13.4 Report Schema

AnachronismReport:
  artifact_id
  referenced_item
  claimed_date
  earliest_valid_date
  severity
  allowed_explanation

status:
  - valid
  - valid_because_forged
  - valid_because_later_copy
  - valid_because_interpolated
  - valid_because_misdated
  - valid_because_ritual
  - invalid_generation_bug

---

14. Language Drift Strategy

Language drift should affect names, titles, ritual terms, place names, legal formulae, scripts, translation uncertainty, and scholar disagreement.

14.1 Language Schema

Language:
  language_id
  parent_language_id
  active_date_range
  regions
  phonology_rules
  morphology_rules
  lexicon
  script_systems
  semantic_shift_rules

14.2 Dialect Snapshot

DialectSnapshot:
  dialect_id
  language_id
  region_id
  date_range
  sound_changes
  spelling_conventions
  grammar_features
  known_terms
  taboo_terms
  loanwords
  semantic_shifts

14.3 Place Name History

```
PlaceNameHistory:
  place_id
  names:
   - name
     language_id
     date_range
     political_context
     religious_context
     derogatory_flag
     archaic_flag
```

14.4 Drift Algorithm

1. Identify populations affected by migration, conquest, trade, ritual isolation, schism, or ecological barrier.
2. Apply sound and spelling changes by region and era.
3. Apply semantic shifts to culturally important terms.
4. Add loanwords from trade and conquest.
5. Split dialects when populations become isolated.
6. Rename places and offices after political or religious changes.
7. Mark old terms as archaic, sacred, taboo, vulgar, obsolete, or technical.
8. Generate mistranslation traps for later interpreters.

14.5 Mistranslation Trap Schema

```
MistranslationTrap:
  source_term
  original_meaning
  later_meaning
  affected_dates
  affected_languages
  likely_wrong_translation
  contradiction_risk
```

---

15. Originality and Trope-Avoidance Strategy

The originality system must detect more than surface resemblance. It should measure structural dependency.

15.1 Core Question

For every cultural feature, artifact, institution, or myth:

Could this only belong to this civilization?

If not, revise it.

15.2 Cultural Feature Schema

```
CulturalFeature:
  feature_id
  feature_type
  description
  solves_problem
  depends_on:
    - ecology_pressure
    - taboo_pressure
    - language_pressure
    - political_pressure
    - material_pressure
    - religious_pressure
    - historical_trauma
  trope_similarity_score
  civilization_specificity_score
```

15.3 Specificity Questions

For every generated feature, ask:

What local problem does this solve?
What local taboo shaped it?
What historical wound does it preserve?
What ecological condition made it necessary?
What language drift changed its meaning?
What evidence would it leave behind?
How could later scholars misunderstand it?
Could this belong anywhere else?

15.4 Trope Risk Detectors

Flag patterns that resemble:

- Roman senate, legions, emperors,
- Egyptian pharaohs, pyramids, sun cults,
- Chinese imperial bureaucracy and mandate models,
- Mesopotamian city-state/ziggurat analogues,
- medieval European feudalism,
- generic Atlantis,
- generic crystal technology,

- generic chosen-one prophecy,
- generic good-gods vs evil-gods pantheon,
- generic empire collapsed by hubris.

15.5 Originality Rule

```
if civilization_specificity_score < threshold:
    revise_feature_until_it_depends_on_local_conditions()
```

Strangeness must be causally grounded. Random weirdness is not originality.

---

16. Discovery and Reinterpretation Algorithm

16.1 Discovery Schema

```
Discovery:
  discovery_id
  discovery_date
  artifact_id
  discoverer_id
  site_id
  public_visibility_date
  initial_classification
  access_tags
```

16.2 Discovery Pipeline

When an artifact is discovered:

1. Select or generate artifact from hidden truth.
2. Validate artifact against hidden truth.
3. Run anachronism detection.
4. Extract public claims.
5. Link claims to existing claims and mysteries.
6. Detect contradictions.
7. Assign contradiction causes.
8. Apply mystery preservation rules.
9. Update public archive graph.
10. Notify interpreters with access to the artifact.
11. Recompute affected theories.
12. Record theory revisions.
13. Generate public-facing discovery summary.

16.3 Mystery-Aware Discovery Algorithm

```
function process_discovery_with_mystery_policy(discovery, archive_state):
  artifact = discovery.artifact
  linked_mysteries = find_related_mysteries(artifact, archive_state)

  for mystery in linked_mysteries:
    effect = classify_discovery_effect(artifact, mystery)

    if effect would violate mystery.preservation_policy:
      artifact = adjust_artifact_interpretation_not_truth(
        artifact,
        mystery
      )

    if mystery.reveal_policy == fully_resolvable:
      update_toward_resolution(mystery, artifact)

    else if mystery.reveal_policy == partially_resolvable:
      increase_confidence_but_cap_resolution(mystery, artifact)

    else if mystery.reveal_policy == never_fully_resolvable:
      generate_new_interpretive_branch(mystery, artifact)

    else if mystery.reveal_policy == contradictory_by_design:
      preserve_multiple_valid_readings(mystery, artifact)

  update_interpreters(artifact, linked_mysteries)
```

16.4 Possible Discovery Effects

Effect	Meaning
confirms_theory	Supports an existing interpretation.
undermines_theory	Contradicts a central claim.
reveals_fraud	Exposes anachronism or false provenance.
creates_mystery	Opens an unresolved question.
recontextualizes_evidence	Changes how earlier artifacts are read.
resolves_detail	Clarifies minor fact without solving central mystery.
deepens_ambiguity	Adds strong evidence for incompatible readings.
reveals_false_resolution	Shows that an old answer was too clean.
changes_question	Reframes what scholars thought the problem

Effect	Meaning
	was.
protects_absence	Confirms that missing evidence matters.

17. Access-Control Model

17.1 Access Levels

Level	Can See
public	Discovered artifacts, public interpretations, unresolved debates.
scholar	More metadata, provenance estimates, contradiction summaries.
curator	Restricted artifacts, suspected forgeries, private catalog notes.
canon	Hidden truth, true provenance, true contradiction causes.
debug	Hidden truth plus validation logs, generation traces, and generation bugs.

17.2 Query Behavior by Access Level

Query	Public	Scholar	Curator	Canon	Debug
What really happened?	Evidence-based uncertainty	Best-supported theory	Restricted notes	Actual truth if knowable	Truth plus trace
Which artifacts are forged?	Publicly exposed forgeries	Accusations and evidence	Suspicions and catalog flags	Actual forgeries	Actual forgeries plus validation logic
What contradictions remain?	Public disputes	Classified contradiction types	Internal causes where allowed	Hidden resolution	Full cause and bug ledger

17.3 Access Filter Pseudocode

```
function answer_query(query, user_access_level):
    raw_answer = resolve_query_against_full_state(query)
    visible_answer = filter_by_access(raw_answer, user_access_level)
    visible_answer = remove_hidden_truth_leaks(visible_answer)
    citations = collect_visible_citations(visible_answer, user_access_level)
    return attach_citations(visible_answer, citations)
```

Hard rule:

Public answers cannot reveal hidden truth unless supported by public evidence.

---

18. Pseudocode

18.1 Initialize Engine

```
function initialize_archive_engine(config):
    rng = create_deterministic_rng(config.seed)

    hidden_truth = generate_hidden_world(config, rng)
    validate_hidden_truth(hidden_truth)

    public_archive = initialize_empty_public_archive()
    artifact_pool = generate_initial_artifact_pool(hidden_truth, config, rng)

    for artifact in artifact_pool:
        validate_artifact_against_truth(artifact, hidden_truth)

    interpreters = initialize_interpreters(config, rng)
    mysteries = initialize_mysteries(hidden_truth, config, rng)

    return ArchiveEngineState(
        config=config,
        hidden_truth=hidden_truth,
        public_archive=public_archive,
        artifact_pool=artifact_pool,
        interpreters=interpreters,
        mysteries=mysteries,
        rng_state=rng.state
    )
```

18.2 Generate Hidden World

```
function generate_hidden_world(config, rng):
    world = new HiddenTruthGraph()

    geography = generate_geography(config, rng)
    ecology = generate_ecology(geography, config, rng)
    proto_culture = generate_proto_culture(geography, ecology, config, rng)

    world.add(geography)
    world.add(ecology)
    world.add(proto_culture)
```

```
    eras = generate_eras(config.time_span, config.historical_density, rng)

    for era in eras:
        candidate_events = propose_events(world, era, config, rng)

        for event in candidate_events:
            if event_is_causally_valid(event, world):
                apply_event_to_hidden_world(event, world)
            else:
                repair_or_reject_event(event, world, rng)

        evolve_languages(world, era, rng)
        evolve_borders(world, era, rng)
        evolve_religions(world, era, rng)
        evolve_trade_routes(world, era, rng)
        update_originality_pressure(world, config)

    validate_hidden_truth(world)
    return world
```

18.3 Validate Event

```
function event_is_causally_valid(event, world):
    for cause_id in event.causes:
        cause = world.get_event(cause_id)
        if cause.end_date > event.start_date:
            return false

    for participant_id in event.participants:
        participant = world.get_entity(participant_id)
        if not exists_during(participant, event.date_range):
            return false

    for technology_id in event.required_technologies:
        technology = world.get_entity(technology_id)
        if not available_during(technology, event.date_range):
            return false

    for location_id in event.locations:
        location = world.get_entity(location_id)
        if not valid_location_during(location, event.date_range):
            return false

    return true
```

18.4 Generate Artifact

```
function generate_artifact(hidden_truth, request, rng):
  artifact_type = select_artifact_type(request, rng)

  source_events = select_hidden_truth_sources(
    hidden_truth,
    request.topic,
    request.date_range,
    request.location
  )

  creator = select_creator_or_attributed_creator(
    hidden_truth,
    source_events,
    artifact_type,
    rng
  )

  true_creation_date = choose_true_creation_date(
    source_events,
    artifact_type,
    creator,
    rng
  )

  claimed_creation_date = choose_claimed_creation_date(
    true_creation_date,
    artifact_type,
    request,
    rng
  )

  distortion_profile = choose_distortion_profile(
    artifact_type,
    creator,
    true_creation_date,
    claimed_creation_date,
    rng
  )

  language_snapshot = select_language_snapshot(
    hidden_truth,
    creator,
```

```
    true_creation_date
)

raw_claims = generate_claims_from_truth(
    source_events,
    creator,
    distortion_profile,
    hidden_truth,
    rng
)

public_text = render_artifact_voice(
    raw_claims,
    language_snapshot,
    artifact_type,
    distortion_profile,
    rng
)

artifact = build_artifact_record(
    artifact_type,
    creator,
    true_creation_date,
    claimed_creation_date,
    language_snapshot,
    public_text,
    raw_claims,
    distortion_profile
)

validation_report = validate_artifact_against_truth(artifact, hidden_truth)

if validation_report.has_unexplained_anachronism:
    return handle_invalid_artifact(artifact, validation_report, request)

if validation_report.has_unexplained_contradiction:
    return handle_invalid_artifact(artifact, validation_report, request)

artifact.reliability_score = compute_reliability(artifact, validation_report)
return artifact
```

18.5 Validate Artifact

```
function validate_artifact_against_truth(artifact, hidden_truth):
    report = new ValidationReport()
    referenced_items = extract_references(artifact.public_text, artifact.claims)

    for item in referenced_items:
        availability = get_availability_window(item, hidden_truth)

        if item_not_available_at_claimed_date(item, artifact.claimed_creation_date):
            explanation = find_anachronism_explanation(artifact, item)

            if explanation exists:
                report.add_allowed_anachronism(item, explanation)
            else:
                report.add_unexplained_anachronism(item)

    contradictions = detect_public_contradictions(artifact, hidden_truth)

    for contradiction in contradictions:
        cause = infer_or_assign_contradiction_cause(artifact, contradiction)

        if cause exists:
            report.add_explained_contradiction(contradiction, cause)
        else:
            report.add_unexplained_contradiction(contradiction)

    return report
```

18.6 Update Interpretations

```
function update_interpretations(engine_state, new_artifact, new_claims, contradictions):
    for interpreter in engine_state.interpreters:
        if not can_access(interpreter, new_artifact):
            continue

        visible_evidence = get_visible_evidence(
            engine_state.public_archive,
            interpreter.access_level,
            interpreter.active_date_range
        )

        for theory in interpreter.theories:
            support = compute_support(new_claims, theory)
            conflict = compute_conflict(new_claims, theory)
```

```
        style_effect = apply_epistemic_style(interpreter, new_artifact, theory)
        forgery_suspicion = compute_forgery_suspicion(interpreter, new_artifact)

        new_confidence = update_theory_confidence(
            theory.confidence,
            support,
            conflict,
            style_effect,
            forgery_suspicion
        )

        if new_confidence < theory.abandonment_threshold:
            revise_or_abandon_theory(theory, new_artifact, new_claims)
        else:
            update_theory(theory, new_artifact, new_claims, new_confidence)

    maybe_generate_new_theory(interpreter, visible_evidence)
```

---

19. Complexity Analysis

Let:

N = number of hidden entities
E = number of hidden events
A = number of artifacts
C = number of public claims
I = number of interpreters
T = number of active theories
R = number of references in one artifact
K = average candidate claims matching a new claim
M = number of tracked mysteries

19.1 Hidden World Generation

Operation	Complexity
Generate entities/events	O(N + E)
Validate event dependencies	O(E * average_dependencies)
Detect causal cycles	O(N + E) with graph traversal
Sort timeline events	O(E log E)
Evolve languages by epoch	O(L * epochs)

19.2 Artifact Generation

Operation	Complexity
Select hidden sources	O(log E + S) with date/topic indexes
Extract references	O(R)
Check anachronisms	O(R log N) with availability indexes
Detect contradictions, naive	O(C) per claim
Detect contradictions, indexed	O(K) per claim
Link mysteries	O(log M + related_mysteries) with subject indexes

19.3 Interpretation Update

Naive update:

O(I * T * C)

Indexed update:

O(affected_interpreters * affected_theories * new_claims)

Recommended indexes:

subject_index
predicate_index
date_range_index
location_index
entity_reference_index
artifact_to_theories
claim_subject_to_theories
mystery_to_artifacts
contradiction_to_theories

19.4 Storage Complexity

Hidden truth graph: O(N + E)
Public archive: O(A + C + contradictions + theories + mysteries)
Generation traces: O(A + E) if fully retained

For MVP, generation traces can be compacted to:

seed
generator_version
source_event_ids
distortion_profile
validation_report

20. Failure Modes

Failure Mode	Cause	Mitigation
Random lore soup	Artifacts generated without hidden truth.	Generate hidden causal graph first.
Over-clean canon	Every mystery has a neat hidden answer.	Use layered truth and protected mysteries.
Accidental contradiction	No contradiction cause assigned.	Mandatory contradiction ledger.
Accidental anachronism	Artifact references unavailable term/entity.	Availability index and anachronism validator.
Hidden truth drift	Public reinterpretation mutates canon.	Enforce write isolation.
Bland fantasy encyclopedia	Low originality pressure.	Civilization-specific dependency checks.
Generic artifact voice	All artifacts sound like summaries.	Artifact voice compiler.
Too much chaos	Contradictions overwhelm causality.	Contradiction budget by era and artifact class.
Scholars all agree	Interpreter models too similar.	Epistemic styles and different evidence preferences.
Forgery too obvious	Only superficial anachronisms.	Add motive, partial truth, plausible material context.
Language drift cosmetic only	Names change without semantic consequences.	Model semantic shifts and translation traps.
Deterministic replay fails	Uncontrolled randomness.	Central seeded RNG and generation trace.
Access leaks hidden truth	Query formatter exposes canon metadata.	Access-filtered answer resolver.
False resolution	New discovery solves protected mystery too neatly.	Mystery preservation policy.
Meaningless weirdness	Strangeness not causally grounded.	Local pressure dependency requirement.

20.1 Failure Handling Policy

When a requested artifact would violate hidden history:

Situation	Engine Response
Useful as fraud	Generate as forged.
Impossible due to dating	Generate as later copy, interpolation, or misdated artifact.
Impossible due to language	Use mistranslation, interpolation, or reject.
Impossible due to technology	Mark as forgery, intrusive deposit, or reject.
Impossible due to hidden truth	Explain impossibility for authorized users.
Protected mystery	Refuse clean resolution or provide bounded uncertainty.
No valid mediation	Reject generation.
Validator finds accidental issue	Mark as UnresolvedGenerationBug in debug mode only.

21. Test Cases

21.1 Core Validation Tests

Test	Input	Expected Output	Purpose
Hidden causal order	Event B caused by Event A, but B dated earlier.	Reject B.	Ensures causes precede effects.
Entity lifespan	Dead ruler signs decree after death.	Accept only if ritual, forgery, later copy, or misdate exists.	Tests lifespan and mediation.
Technology availability	Bronze tool before metallurgy.	Reject or mark forged/intrusive.	Tests technology chronology.
Language drift	Later dialect word appears in claimed early inscription.	Explain as copy/interpolation/forgery or reject.	Tests linguistic anachronism.
Place-name drift	Artifact uses later city name too early.	Trigger anachronism report.	Tests name history.
Calendar error	Two artifacts date same eclipse differently.	Cause = calendar conversion error.	Tests valid contradiction.
Forged decree	Old king uses later seal/title.	Accepted as forgery with motive.	Tests forgery model.
Damaged manuscript	Missing phrase creates succession ambiguity.	Cause = damage or genuine uncertainty.	Tests damaged evidence.
Oral history	Song merges three	Cause =	Tests mythic

Test	Input	Expected Output	Purpose
compression	rulers.	mythologized memory.	distortion.
Propaganda chronicle	State text credits ruler falsely.	Cause = propaganda.	Tests political distortion.
Censorship	Later manuscript omits banned sect.	Cause = deliberate censorship.	Tests absence as evidence.
Hidden truth preservation	New discovery contradicts public theory.	Hidden truth unchanged.	Tests canon isolation.
Public access	Public asks what really happened.	Evidence-based uncertainty only.	Tests access filtering.
Canon access	Canon user asks what really happened.	Hidden truth if knowable.	Tests privileged access.
Deterministic replay	Same seed/config twice.	Identical hidden graph and artifacts.	Tests reproducibility.

21.2 New Epistemic and Aesthetic Tests

Test	Input	Expected Output	Purpose
Protected mystery	Artifact appears to solve protected mystery.	Confidence capped or evidence reframed.	Prevents over-explanation.
Ritual contradiction	Dead king signs decree.	Accepted as ritual/legal truth if convention exists.	Tests non-factual validity.
Meaningful absence	Archive lacks banned sect records.	Absence becomes evidence only if censorship/taboo exists.	Tests silence handling.
Artifact voice	Royal decree, trade ledger, ritual song describe same event.	Three distinct voices and claim types.	Prevents generic prose.
Interpreter style	Same forged decree shown to five scholars.	Different plausible interpretations.	Tests epistemic diversity.
Specificity pressure	Generic moon cult generated.	Rejected or transformed.	Tests originality v2.
Mythic truth	Myth contradicts canon but shapes law.	Accepted as culturally true, factually false.	Tests layered truth.
Unknowable loss	User asks canon	Engine reports no	Tests mystery

Test	Input	Expected Output	Purpose
	answer to protected lost event.	recoverable answer exists.	preservation.
Translation uncertainty	Ambiguous term with semantic drift.	Multiple translations with consequences.	Tests linguistic ambiguity.
False resolution	New artifact confirms old theory too neatly.	Forgery/suspicion/ misclassification check triggered.	Tests suspicious neatness.

22. Minimal Implementation Roadmap

22.1 MVP Principle

The MVP should prove the hardest invariant:

> Hidden truth remains coherent while public evidence can be biased, contradictory, access-filtered, and epistemically unstable.

Do not start with a full civilization simulator. Start with a narrow archive slice.

Phase 1 - Core Data Structures

Build:

1. Hidden truth graph.
2. Event records.
3. Entity records.
4. Artifact records.
5. Claim records.
6. Contradiction records.
7. Access tags.
8. Seeded deterministic state.

Acceptance criteria:

- Can create a hidden timeline.
- Can create artifacts linked to hidden events.
- Can answer public vs canonical questions differently.

Phase 2 - Validation Layer

Build:

1. Temporal availability checks.
2. Entity lifespan checks.

3. Technology availability checks.
4. Place-name availability checks.
5. Contradiction cause enforcement.
6. Deterministic replay tests.

Acceptance criteria:

- Invalid artifacts are rejected or explained.
- Every contradiction has a cause.
- Same seed reproduces same state.

Phase 3 - Artifact Simulator MVP

Support five artifact types first:

1. inscription,
2. damaged manuscript,
3. forged decree,
4. oral history,
5. trade ledger.

Acceptance criteria:

- Each artifact has required metadata.
- Each artifact produces claims.
- Each artifact has reliability scoring.
- Forged artifacts can contain justified anachronisms.

Phase 4 - Artifact Voice Compiler

Build:

1. voice profiles,
2. formula rules,
3. damage rendering,
4. translation apparatus,
5. taboo substitution,
6. catalog bias.

Acceptance criteria:

- Different artifact classes produce distinct textual styles.
- Artifact voice depends on civilization-specific pressures.
- Public text does not collapse into neutral lore summary.

Phase 5 - Interpreter Engine

Build three interpreter types first:

1. archaeologist,
2. religious authority,
3. political propagandist.

Then add epistemic styles:

1. stratigraphic literalist,
2. ritual formalist,
3. philological maximalist.

Acceptance criteria:

- Same evidence can produce divergent theories.
- Theories cite artifacts.
- New discoveries update theories.

Phase 6 - Language Drift MVP

Build:

1. place-name history,
2. title history,
3. simple sound/orthographic drift,
4. semantic mistranslation traps.

Acceptance criteria:

- Artifacts use historically valid terms.
- Late terms in early texts trigger anachronism detection.
- Mistranslations can create valid contradictions.

Phase 7 - Mystery Preservation MVP

Build:

1. Mystery records.
2. Reveal policies.
3. Protected questions.
4. Confidence caps.
5. Discovery effects that deepen rather than resolve ambiguity.

Acceptance criteria:

- Some mysteries can be partially resolved.
- Some mysteries remain deliberately unresolved.
- The engine can distinguish mystery from generation bug.

Phase 8 - Originality Pressure MVP

Build:

1. trope vector schema,
2. civilization-specific dependency scoring,
3. similarity scoring,
4. mutation rules,
5. rejection threshold.

Acceptance criteria:

- Direct analogues are flagged.
- Mutated replacements remain causally coherent.
- Generated features depend on local pressures.

Phase 9 - Discovery/Reinterpretation Loop

Build:

1. discovery events,
2. public visibility dates,
3. theory revision history,
4. evidence recontextualization.

Acceptance criteria:

- New artifact updates public archive.
- Hidden truth does not change.
- Theories revise with citations.

Phase 10 - Query Interface

Support:

What really happened?
What would scholar X believe in year Y?
What evidence supports this theory?
Which artifacts are forged?
What contradictions remain unresolved?
Generate a museum exhibit.
Generate a newly discovered inscription.
Generate a disputed translation.

Acceptance criteria:

- Answers obey access level.
- Answers cite visible artifacts.

- Hidden truth is not leaked to public users.

---

23. Recommended Implementation Shape

```
/impossible_archive
  /core
    hidden_truth_graph
    layered_truth_model
    public_archive_graph
    event_model
    entity_model
    artifact_model
    claim_model
    contradiction_model
    mystery_model

  /generation
    world_generator
    event_generator
    artifact_generator
    artifact_voice_compiler
    language_drift
    originality_pressure

  /validation
    causality_validator
    anachronism_detector
    contradiction_validator
    mystery_policy_validator
    access_leak_validator

  /interpretation
    interpreter_model
    epistemic_style_model
    theory_model
    evidence_scorer
    reinterpretation_engine

  /query
    query_classifier
    answer_resolver
    access_filter
```

citation_builder

/tests
causal_consistency_tests
artifact_provenance_tests
anachronism_tests
interpretation_update_tests
access_control_tests
deterministic_replay_tests
mystery_preservation_tests
artifact_voice_tests
originality_pressure_tests

---

24. Open Questions Before Coding

24.1 Storage and Runtime

1. Should the first implementation be an in-memory prototype, local file-backed engine, or database-backed engine?
2. Should hidden truth be stored as JSON, SQLite, graph database, relational tables, or custom serialized state?
3. Is the target use case writing assistance, game runtime, procedural worldbuilding tool, collaborative archive simulation, or interactive fiction?

24.2 Generation Scale

4. What initial scale should the MVP target?

small: 100 years, 50 events, 30 artifacts
medium: 800 years, 500 events, 300 artifacts
large: 3000 years, 5000 events, 3000 artifacts

5. Should artifacts be generated eagerly, lazily, or hybrid?

Recommended: **hybrid**. Generate artifact origins and probabilities early, but render public text lazily.

24.3 Language System

6. Should language drift be lightweight name/title drift, medium phonological and semantic drift, or deep constructed-language simulation?

Recommended for MVP: **medium-light**.

24.4 Interpretation System

7. Should scholar theories be rule-based, LLM-assisted, probabilistic/Bayesian, argument-graph based, or hybrid?

Recommended: **argument graph plus weighted scoring, with optional LLM rendering later**.

24.5 Access Control

8. Is hidden truth access intended for developer only, game master, privileged user, or unlockable gameplay layer?

24.6 Originality Constraints

9. Should the trope detector use hand-authored rules, embedding similarity, LLM critique, or all three?

Recommended: **hand-authored rules first**, optional critique later.

24.7 Output Style

10. Should generated artifacts be written in academic style, museum label style, poetic translation style, fragmentary catalog style, or selectable modes?

Recommended: selectable modes, but with artifact voice constraints always applied underneath.

---

25. Final Core Principle

The Impossible Archive Engine should obey this rule:

> The archive is not a distorted window onto truth. It is a machine for producing historically situated relationships between truth, evidence, ritual, power, memory, fraud, loss, and interpretation.

A weak implementation will generate random lore.

A competent implementation will generate a coherent fictional historiography.

The target implementation should simulate the unstable boundary between:

- what happened,
- what was remembered,
- what was written,
- what survived,
- what was forged,
- what was forbidden,
- what was translated,
- what was ritually true,
- what scholars believe,
- and what can never be recovered.

That is the final architecture target.